

Proposta de Arquitetura AWS

Pipeline automatizada de observabilidade e SRE em ambiente distribuído

Preparado por: Laís Caroline Pinto Moreira | Agosto de 2026

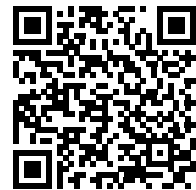


Diagrama online

1. Resumo executivo

A proposta é construir uma plataforma gerenciada e orientada a eventos para coletar, padronizar, correlacionar e consolidar logs, métricas, traces e eventos provenientes de aplicações distribuídas. A solução evita forçar sinais diferentes em um único armazenamento: cada sinal utiliza o backend mais adequado, enquanto identificadores comuns e uma visão central permitem a análise ponta a ponta.

Premissa de escopo

Considerando os conhecimentos solicitados na vaga - observabilidade, SRE, sistemas distribuídos, APIs REST, JSON e ferramentas de monitoramento -, o termo ambiente mesh é interpretado como um ecossistema de microsserviços ou service mesh. Caso o termo represente Data Mesh analítico, a camada histórica S3/Glue/Athena pode ser ampliada sem alterar o núcleo de observabilidade.

2. Objetivos da solução

Consolidação operacional

Criar um contrato comum de telemetria com service.name, environment, trace_id e correlation_id, permitindo unir eventos relacionados sem perder a especialização dos backends.

Troubleshooting ponta a ponta

Correlacionar logs, métricas e traces para localizar rapidamente o serviço, a dependência e a etapa responsáveis por degradação ou erro.

Confiabilidade mensurável

Definir Golden Signals, SLIs e SLOs por jornada crítica, reduzindo alertas sem ação e priorizando impacto real ao usuário.

Resposta rastreável

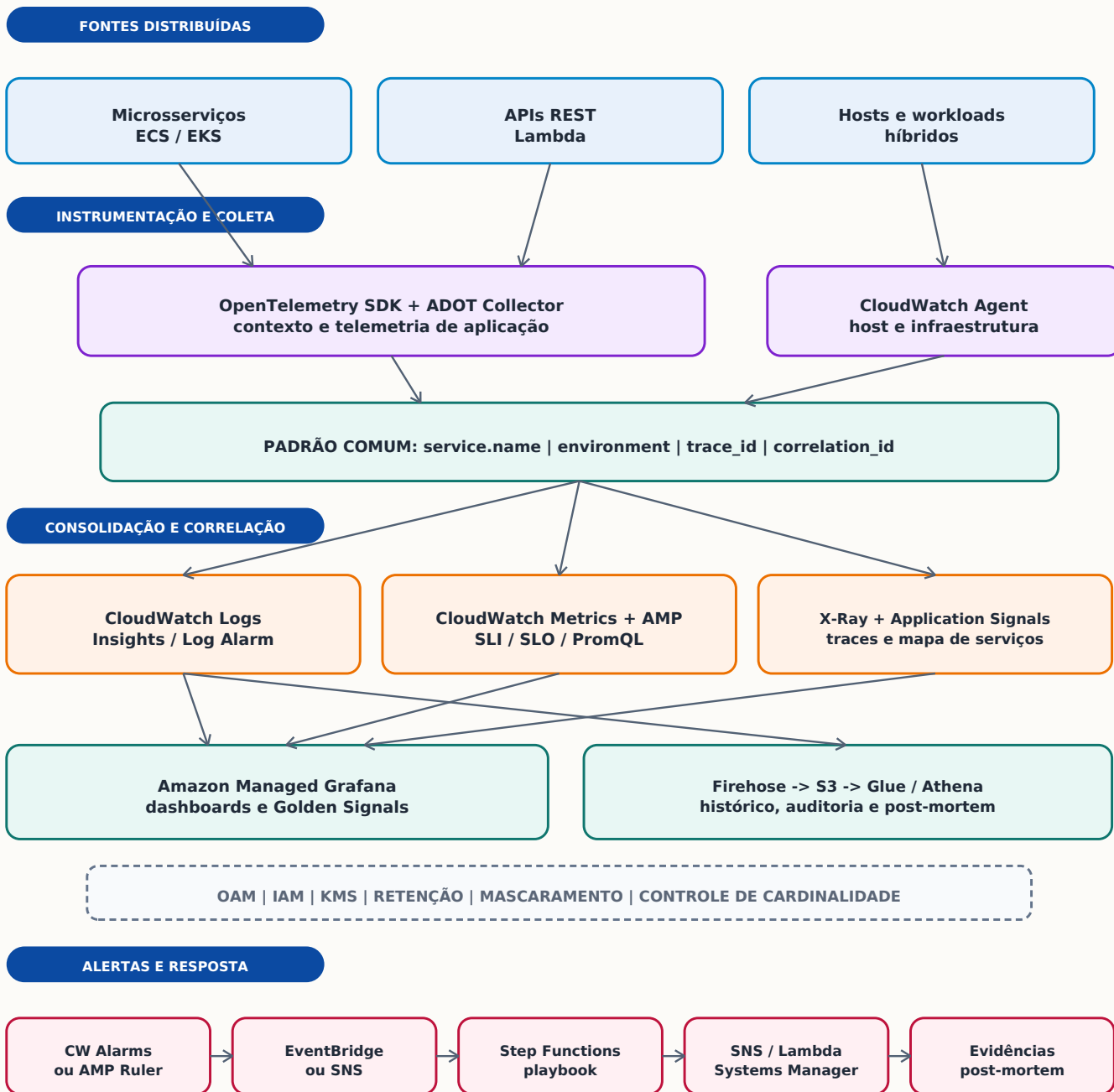
Automatizar notificações, coleta de evidências e remediações seguras por playbooks com retries, controle de acesso e registro de execução.

Resultado esperado

Uma visão central da saúde dos serviços, com redução do tempo médio de detecção e recuperação, maior qualidade dos post-mortems e expansão progressiva por domínio.

3. Arquitetura proposta

A pipeline separa cinco responsabilidades: fontes, instrumentação, consolidação, consumo e resposta. A camada de governança atravessa todo o fluxo.



Consolidar significa correlacionar sinais por contexto comum e oferecer uma visão central, preservando o backend adequado a cada sinal.

Princípio de consolidação

Consolidar não significa armazenar tudo no mesmo mecanismo. Significa padronizar o contexto, correlacionar sinais, oferecer consulta central e preservar o backend tecnicamente adequado para logs, métricas e traces.

4. Componentes AWS e conexões

A tabela abaixo explicita o papel de cada componente e elimina ambiguidades sobre o encadeamento técnico.

Camada	Serviços AWS	Conexão e responsabilidade
Instrumentação	OpenTelemetry SDK	Insere contexto de trace e atributos padronizados nas aplicações e APIs.
Coleta	ADOT Collector + CloudWatch Agent	Recebem telemetria via OTLP. ADOT prioriza aplicação; o Agent complementa host, contêiner e infraestrutura.
Logs	CloudWatch Logs + Logs Insights	Centraliza logs estruturados. Log Alarm ou metric filter transforma padrões de erro em sinais acionáveis.
Métricas	CloudWatch Metrics + AMP	Armazena métricas técnicas e séries Prometheus; sustenta Golden Signals, SLIs e SLOs.
Tracing	X-Ray + Application Signals	Correlaciona spans, latência e dependências; oferece mapa de serviços e visão de confiabilidade.
Visão central	Managed Grafana + Observability Access Manager (OAM)	Grafana consulta os backends; o OAM centraliza observabilidade entre contas e domínios.
Histórico	Firehose + S3 + Glue + Athena	Mantém histórico de longo prazo para auditoria, tendências, troubleshooting e post-mortems.
Automação	EventBridge + Step Functions	Roteia alertas e executa playbooks rastreáveis, com aprovação humana quando necessário.
Ações	SNS + Lambda + Systems Manager	Notifica a equipe, coleta evidências e executa remediações seguras e idempotentes.
Segurança	IAM + KMS + CloudTrail	Aplica privilégio mínimo, criptografia, auditoria e separação entre contas e ambientes.

5. Rotas de alerta e automação

CloudWatch Logs usam Log Alarm ou metric filter; métricas usam CloudWatch Alarms. Mudanças de estado são enviadas ao EventBridge, que inicia a Step Function de resposta.	Prometheus Métricas no AMP usam regras do Ruler e Alertmanager. O Alertmanager agrupa, deduplica e encaminha alertas ao SNS, evitando tratar o AMP como origem direta de CloudWatch Alarm.
---	--

Playbook seguro

A Step Function primeiro coleta evidências e verifica contexto. Depois notifica o plantão e, somente para cenários previamente aprovados, executa Lambda ou Systems Manager. Ações destrutivas ou de alto risco exigem aprovação humana.

6. Observabilidade orientada à confiabilidade

Latência p50, p95 e p99 por serviço, operação e dependência crítica. SLI sugerido: proporção de requisições abaixo do limite de latência.	Tráfego Requisições por segundo, throughput e volume por consumidor, endpoint, domínio e ambiente.
Erros Taxa de respostas 5xx, exceções, timeouts, retries e mensagens em DLQ, sempre correlacionadas ao impacto no usuário.	Saturação CPU, memória, conexões, filas, concorrência e limites de serviço. Alertas devem considerar tendência e janela, não apenas pico isolado.

7. Contrato mínimo de telemetria

Um contrato JSON mínimo permite correlação e roteamento consistentes entre domínios. Dados sensíveis devem ser mascarados antes da ingestão.

```
{
  "timestamp": "2026-08-26T12:00:00Z",
  "service_name": "payments-api",
  "environment": "prod",
  "trace_id": "4bf92f...",
  "correlation_id": "req-8f21...",
  "severity": "ERROR",
  "http_status_code": 500,
  "duration_ms": 842
}
```

8. Segurança, resiliência e custo

Segurança e governança IAM com privilégio mínimo, KMS em repouso, TLS em trânsito, CloudTrail para auditoria, segregação por conta/ambiente e políticas de retenção. Tokens, CPF e dados financeiros não devem entrar em logs.	Confiabilidade da própria pipeline Retries com backoff, idempotência, DLQ quando aplicável, alarmes sobre falhas de coleta e execução, amostragem de traces e limites de cardinalidade para controlar custo.
--	--

9. Plano de implementação do MVP

Fase 1 - Descoberta e contrato Selecionar 2 ou 3 serviços críticos, mapear jornadas, definir campos JSON, retenção, classificação de dados e SLO inicial.	Fase 2 - Instrumentação Aplicar OpenTelemetry, configurar ADOT/CloudWatch Agent e validar logs, métricas e traces com correlação ponta a ponta.
Fase 3 - Visualização e alerta Criar dashboard de Golden Signals, alarmes acionáveis e roteamento correto para EventBridge ou Alertmanager/SNS.	Fase 4 - Resposta e evolução Implementar um playbook, simular incidente, medir resultados, registrar post-mortem e expandir para novos domínios.

10. Critérios de aceite

- Telemetria dos serviços piloto contém os campos obrigatórios e pode ser correlacionada por trace_id ou correlation_id.
- Dashboard apresenta latência, tráfego, erros e saturação por serviço e ambiente.
- Um incidente simulado gera alerta, roteamento, notificação, coleta de evidências e registro da execução.
- A equipe consegue navegar do indicador degradado para trace e logs associados sem troca manual de contexto.
- Políticas de acesso, retenção, mascaramento, amostragem e cardinalidade são validadas antes da produção.

11. Conclusão

Proposta final

A arquitetura responde ao desafio ao automatizar a agregação e a consolidação da telemetria distribuída, conectando coleta, correlação, visualização, alertas e resposta. O desenho é incremental, governado e mensurável por SLOs, permitindo iniciar com um MVP e evoluir conforme criticidade, volume e maturidade do time.

12. Referências oficiais

- [CloudWatch Agent - métricas, logs e traces](#)
- [OpenTelemetry no Amazon CloudWatch](#)
- [CloudWatch Application Signals](#)
- [CloudWatch Alarm e EventBridge](#)
- [Alertmanager no Amazon Managed Service for Prometheus](#)
- [Fontes de dados do Amazon Managed Grafana](#)
- [CloudWatch cross-account observability](#)

Diagrama online: <https://laismoreira07.github.io/ict-case-arquitetura-aws/>